Gen AI Driven Wellbeing Coach and Health Management Platform

Software Requirements Specification

Team 48

Rishabh Goyal, Aditya Singh, Tanush Garg, Avnish Uba, Sashi Kumar Gudipati

1 Brief Problem Statement

Health data is fragmented across wearable apps and medical reports (often PDFs), making it hard for individuals to understand their health status and take timely action. Clinicians also lack a lightweight way to monitor patients between visits without wading through raw, unstructured data. This project builds a platform that ingests medical reports, extracts verifiable facts, and provides explainable alerts and AI-assisted guidance grounded in the user's own evidence.

2 System Requirements (Technologies/Languages/Libraries)

- **Backend:** Python 3, FastAPI, Uvicorn
- **Storage + DB:** Supabase (PostgreSQL + Storage), Supabase Python client
- **OCR + PDF processing:** Tesseract OCR (`pytesseract`), `pdf2image`, Pillow, OpenCV, NumPy
- **OS dependencies:** Tesseract installed and Poppler available for PDF-to-image conversion
- **Text preprocessing:** OCR text cleaning + chunking utilities; LangChain text splitters (for recursive chunking)
- **Embeddings (RAG):** `SentenceTransformers` with model `BAAI/bge-base-en-v1.5`
- **Configuration:** Environment variables + `.env` support (e.g., `SUPABASE_URL`, `SUPABASE_SERVICE_ROL` bucket/table names)
- **(Planned for R2 / integration):** External context APIs (weather/AQI), LLM reasoning layer (Gemini) for coached answers with citations

3 Users Profile

- **Patient (primary):** Uploads medical reports; views extracted labs, alerts, and AI coach explanations. Comfortable using a web/mobile UI, not necessarily medically trained.

- **Doctor/Clinician (secondary):** Reviews a list of patients and risk status; drills into evidence-backed alerts and summaries. High computer proficiency; expects concise, explainable outputs.
- **System Admin/Developer (support):** Configures Supabase, environment variables, and monitors ingestion/OCR failures. Technical user.

4 Requirements

4.1 Feature Requirements (Use Cases)

Feature / Use Case	Description	Release
Upload medical report	Patient uploads a PDF/image report which is stored under the patient's scope in secure storage; system returns a reference path/URL.	R1
View uploaded report metadata	Patient can view basic metadata for an uploaded report (filename, upload timestamp, link/reference).	R1
Run OCR on a stored report	Patient triggers OCR; system downloads the stored file, performs OCR, and persists OCR text + confidence.	R1
View OCR output	Patient can view OCR text and an OCR confidence/quality indicator for the report.	R1
Extract structured lab results	Patient triggers deterministic (regex-based) extraction of lab measurements from OCR text and stores them as atomic facts linked to the report.	R1
View structured lab results	Patient can list lab results for a report including value, unit, reference range, and source page when available.	R1
Clean OCR text for retrieval	System cleans noisy OCR text to remove repeated headers/junk and produce meaningful lines for downstream retrieval.	R1
Chunk OCR text	System splits cleaned text into retrieval-ready chunks (table-like rows + narrative chunks) to support evidence retrieval.	R1
Embed chunks	System generates vector embeddings for chunks using a configured embedding model for semantic search.	R1
Retrieve evidence for a question	Patient asks a question; system retrieves top relevant chunks with source metadata (file + page) as citations.	R1
AI coach answer with citations	Patient receives an answer that is grounded in retrieved evidence and returns citations/snippets for transparency.	R1

Continued on next page

Feature / Use Case	Description	Release
Generate explainable alerts	System produces deterministic alerts (low/medium/high) from extracted facts and links each alert to evidence (report/lab/snippet).	R1
View dashboard summary	Patient views a dashboard with wellbeing score/trend (and optionally environment context) plus active alert counts.	R1
Wearable data sync & automated alert generation	Continuously ingests background wearable vitals and evaluates them against rules, patient baselines, and environmental data to dynamically flag anomalies.	R2
RAG-powered AI voice assistant	Allows patients to ask natural language voice questions about their health using a RAG pipeline grounded in their vectorized medical history.	R2
Doctor priority dashboard & patient monitoring	Equips doctors with a priority-sorted roster, aggregated AI clinical timelines, and manual alert reevaluation capabilities.	R2

4.2 Use Case Diagram

Actors to include: Patient, Doctor, System Admin/Developer. External systems: Supabase Storage/DB, OCR engine (Tesseract), Weather/AQI API, LLM provider.

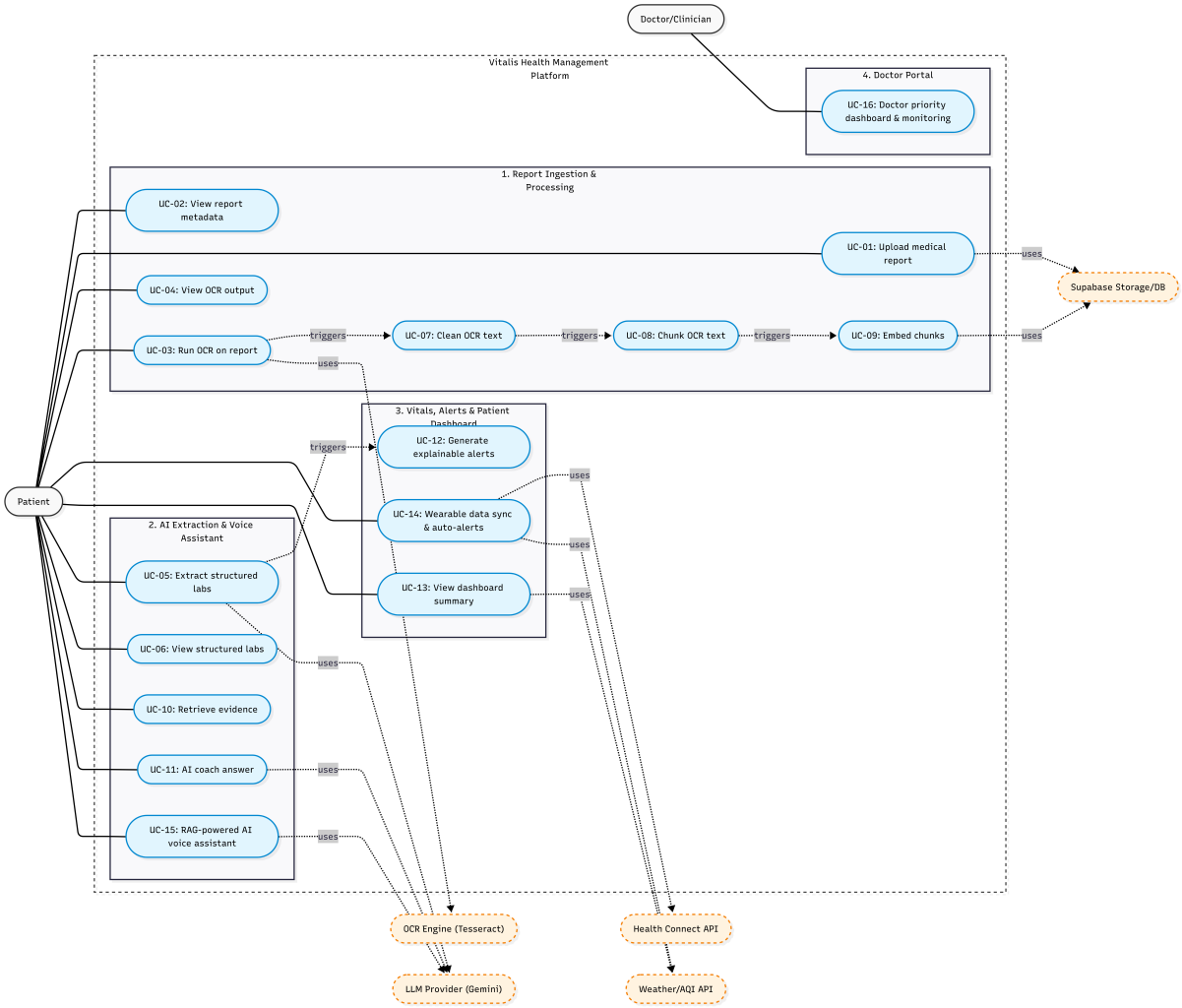

Figure 1: System Use Case Diagram

5 Use Case Descriptions

UC-01: Upload medical report

Overview: Patient uploads a medical report (PDF/image). The system stores the file in secure storage under the patient scope and returns a reference.
Actors: Patient
Pre-condition: Patient has a valid user identifier; system storage is configured.
Flow:

- **Main (success) Flow:**

  1. Patient selects a report file and submits upload.
  2. System validates non-empty file and required user identifier.
  3. System generates a unique, user-scoped storage path.
  4. System uploads bytes to storage with the correct content type.

5. System returns storage reference (path) and accessible URL (public/signed per deployment).

- **Alternate Flows:**
  - *A1.* Missing user identifier → System rejects request with validation error. Post-condition: no file stored.
  - *A2.* Empty/corrupt file → System rejects request. Post-condition: no file stored.
  - *A3.* Storage failure → System returns error. Post-condition: no file stored.

Post-Condition: Report file is stored and uniquely addressable; patient receives reference.

UC-02: View uploaded report metadata

Overview: Patient views metadata for an uploaded report to confirm it is available for processing.
Actors: Patient
Pre-condition: Patient has uploaded at least one report.
Flow:

- **Main Flow:**
  1. Patient opens "My Reports".
  2. System lists reports with filename, upload time, and processing status (uploaded/ocr_done/extrac

- **Alternate Flows:**
  - *A1.* No reports → System shows empty state.

Post-Condition: Patient can identify which report to OCR/extract.

UC-03: Run OCR on a stored report

Overview: Patient triggers OCR on a previously uploaded report. System performs OCR and persists OCR text and confidence.
Actors: Patient, System (OCR engine)
Pre-condition: Report exists in storage; OCR dependencies available.
Flow:

- **Main Flow:**
  1. Patient selects a report and starts OCR.
  2. System downloads the file bytes from storage.
  3. If PDF, system converts pages to images.
  4. System preprocesses images for OCR quality.
  5. System runs OCR and computes average confidence.
  6. System stores OCR text + confidence and source metadata in the database.

- **Alternate Flows:**
  - *A1.* Storage path invalid/not found → System returns "report not found".
  - *A2.* PDF conversion fails (missing Poppler, corrupt PDF) → System returns OCR error; status marked failed.
  - *A3.* OCR engine fails → System returns OCR error; status marked failed.

Post-Condition: OCR output and confidence are stored for the report (or failure is recorded).

UC-04: View OCR output

Overview: Patient reviews OCR text and confidence to validate scan quality before extraction/AI queries.
Actors: Patient
Pre-condition: OCR has been completed for the report.
Flow:

- **Main Flow:**
  1. Patient opens report OCR results.
  2. System displays OCR text and confidence indicator.
- **Alternate Flows:**
  - *A1.* OCR not available → System asks patient to run OCR first.

Post-Condition: Patient can confirm OCR readiness.

UC-05: Extract structured lab results

Overview: Patient triggers deterministic extraction of lab values from OCR text and stores each lab as an atomic fact linked to the report.
Actors: Patient, System
Pre-condition: OCR text exists for the report.
Flow:

- **Main Flow:**
  1. Patient clicks "Extract Labs".
  2. System loads OCR text for the report.
  3. System applies allow-listed test name matching and numeric/unit parsing.
  4. System stores extracted lab results linked to the report.
- **Alternate Flows:**
  - *A1.* Report not found → System returns error; no rows inserted.
  - *A2.* No extractable labs → System returns success with 0 inserted.
  - *A3.* DB insert fails → System returns error; partial inserts handled per transaction strategy.

Post-Condition: Lab results are stored (or the system reports that none were confidently extractable).

UC-06: View structured lab results

Overview: Patient views extracted labs with units, reference ranges, and traceability (source page/snippet when available).
Actors: Patient
Pre-condition: Labs have been extracted for the report.
Flow:

- **Main Flow:**
  1. Patient opens the "Labs" view for a report.
  2. System lists lab results with test name, numeric value, unit, reference range, and provenance.
- **Alternate Flows:**
  - *A1.* No labs extracted yet → System prompts to run extraction.

Post-Condition: Patient can inspect verifiable facts extracted from the report.

UC-07: Clean OCR text for retrieval

Overview: System transforms raw OCR into cleaned lines by removing repeated headers/junk and preserving medically relevant content.
Actors: System
Pre-condition: OCR text exists.
Flow:

- **Main Flow:**
  1. System splits raw text into line-like segments.
  2. System removes known junk patterns (headers, page markers, boilerplate).
  3. System returns cleaned text suitable for chunking/retrieval.
- **Alternate Flows:**
  - *A1.* OCR text empty → System returns empty cleaned result.

Post-Condition: Cleaned text is produced without altering the stored immutable OCR original.

UC-08: Chunk OCR text

Overview: System produces retrieval-ready chunks (tabular rows + narrative chunks) for semantic search and citation.
Actors: System
Pre-condition: Cleaned OCR text exists.
Flow:

- **Main Flow:**
  1. System detects section-like headings and splits into sections.

2. System extracts measurement-like rows as chunks.
3. System recursively splits remaining narrative text into bounded chunks with overlap.

Post-Condition: A chunk set exists for embedding/retrieval.

UC-09: Embed chunks

Overview: System generates vector embeddings for chunks for semantic retrieval.
Actors: System
Pre-condition: Chunk set exists; embedding model available.
Flow:

- **Main Flow:**
  1. System loads the configured embedding model.
  2. System embeds chunks and stores vectors + required metadata (user/report/source).
- **Alternate Flows:**
  - *A1.* Embedding model not available → System fails gracefully and logs error.

Post-Condition: Chunks are searchable by vector similarity.

UC-10: Retrieve evidence for a question

Overview: Patient asks a health question; system retrieves the most relevant chunks with source metadata for citations.
Actors: Patient, System
Pre-condition: Chunks are embedded for the patient's documents (or a fallback retrieval mode exists).
Flow:

- **Main Flow:**
  1. Patient submits a question.
  2. System embeds the query.
  3. System retrieves top-k relevant chunks limited to the patient scope.
  4. System returns retrieved chunks with source file and page metadata.
- **Alternate Flows:**
  - *A1.* No vectors available → System returns empty citations and suggests uploading/processing reports.

Post-Condition: Evidence set is available for answer generation.

UC-11: AI coach answer with citations

Overview: System generates an answer that is grounded in retrieved evidence and returns citations/snippets.
Actors: Patient, System (LLM)
Pre-condition: Evidence retrieved (UC-10).
Flow:

- **Main Flow:**
  1. System builds a prompt containing the question + retrieved evidence.
  2. LLM generates an answer constrained to evidence.
  3. System returns answer plus citations (source file, page, snippet).
- **Alternate Flows:**
  - *A1.* LLM unavailable → System returns error and still shows retrieved evidence.
  - *A2.* Evidence empty → System returns a safe response asking for more data and avoids unsupported claims.

Post-Condition: Patient receives an answer and can trace it to sources.

UC-12: Generate explainable alerts

Overview: System generates deterministic alerts from extracted facts and stores explicit evidence links for every alert.
Actors: System
Pre-condition: Labs and/or other signals exist.
Flow:

- **Main Flow:**
  1. System evaluates rules over lab results and trends.
  2. System assigns severity (low/medium/high).
  3. System stores alert plus evidence references (report/lab/snippet).
- **Alternate Flows:**
  - *A1.* Insufficient evidence → System does not create alert.

Post-Condition: Alerts exist only when explainable via stored evidence.

UC-13: View dashboard summary

Overview: Patient views a summary of wellbeing score/trend, environment context (if enabled), and active alert counts.
Actors: Patient
Pre-condition: Patient has data (reports/labs/alerts).
Flow:

- **Main Flow:**

1. Patient opens dashboard.
2. System loads computed summary metrics and active alerts count.
3. System displays environment context if available.

- **Alternate Flows:**
  - *A1.* New user/no data → Dashboard shows onboarding guidance and zero-state metrics.

Post-Condition: Patient understands current status at a glance.

UC-14: Wearable data sync & automated alert generation

Overview: This use case continuously ingests background wearable vitals and evaluates them against a deterministic rules engine that factors in the patient's baseline and real-time environmental data (like AQI or temperature). Its broader purpose is to enable proactive, continuous monitoring, immediately flagging critical health anomalies to both the patient and doctor before they become emergencies.
Actors: Patient (via Wearable API / Health Connect), System
Pre-condition: Patient has granted permissions for wearable data syncing.
Flow:

- **Main Flow:**
  1. App batches background vitals (HR, SpO2, Sleep) and POSTs payload to the backend.
  2. System bulk-inserts raw wearable records.
  3. Rules Engine fetches patient baseline and evaluates vitals against clinical thresholds and active conditions.
  4. System fetches real-time environmental data (AQI, Temperature) to dynamically modify severity.
  5. Critical anomaly generates an Alert record and triggers a real-time WebSocket notification.

- **Alternate Flows:**
  - *A1.* Normal Vitals → Rule engine completes evaluation with no anomalies; no alerts are generated.

Post-Condition: Wearable data is securely stored, and any detected health risks are immediately flagged for both the patient and their assigned doctor.

UC-15: RAG-powered AI voice assistant

Overview: This use case allows patients to ask natural language voice questions about their health, using a Retrieval-Augmented Generation (RAG) pipeline to fetch relevant data directly from their vectorized medical history. Its broader purpose is to provide patients with accessible, personalized, and clinically-grounded explanations of their health without the risk of the AI hallucinating outside medical facts.

Actors: Patient, System
Pre-condition: Patient has uploaded medical reports or synced vitals.
Flow:

- **Main Flow:**
  1. Patient speaks a health query into the app microphone.
  2. System uses Speech-to-Text (STT) to transcribe audio into text.
  3. Vector database performs similarity search to retrieve relevant medical report chunks and recent vitals.
  4. LLM generates a personalized, clinically-grounded answer using the retrieved context.
  5. System uses Text-to-Speech (TTS) to convert the response into audio, playing it while displaying text.
- **Alternate Flows:**
  - *A1.* No Relevant Context Found → System informs the patient that there is not enough data in their history to safely answer the question.

Post-Condition: Patient receives an accurate, personalized voice/text explanation grounded strictly in their health data.

UC-16: Doctor priority dashboard & patient monitoring

Overview: This use case equips healthcare professionals with a priority-sorted patient roster and an aggregated, AI-generated clinical timeline for rapid review. Its broader purpose is to streamline clinical triage, allowing doctors to focus instantly on high-risk patients and manually reevaluate active alerts based on the absolute latest lab and wearable data.
Actors: Doctor, System
Pre-condition: Doctor is authenticated with verified RBAC (role == 'doctor') and has assigned patients.
Flow:

- **Main Flow:**
  1. Doctor logs into the Web Dashboard.
  2. System queries roster, sorting patients by risk level (Critical Alerts first).
  3. Doctor selects a high-risk patient profile.
  4. System retrieves historical lab trends and triggers the Summarizer Service for a weekly longitudinal AI summary.
  5. Doctor clicks "Reevaluate Alerts" to trigger an idempotent rule engine run.
  6. System fetches the absolute newest lab results and wearable data, deletes old alerts, inserts newly calculated alerts, and refreshes the UI.
- **Alternate Flows:**

– *A1.* Unauthorized Access → User without doctor role attempts access; System denies access (403 Forbidden).
– *A2.* No Assigned Patients → Dashboard renders an empty state with onboarding guidance.

Post-Condition: Doctor successfully triages at-risk patients and ensures all clinical alerts reflect the latest available data.